

HDOC – Experiment 33

Question

Write a C program to perform the following menu-driven operations using separate functions:

1. Read three variable-length 2D integer arrays A, B, and C, each of dimensions (m x n), and verify the associative property of matrix addition: $(A+B)+C=A+(B+C)$
2. Read three variable-length 2D integer arrays with the following dimensions: A (m x n), B (n x p), C (p x q). Verify the associative property of matrix multiplication: $(A \times B) \times C = A \times (B \times C)$
3. Exit

❖ Algorithm

Main Algorithm

1. Start.
2. Display the menu:
 - Associative property of matrix addition
 - Associative property of matrix multiplication
 - Exit
3. Read the user's choice.
4. Use switch:
 - Choice 1 → call `associativeAddition()`.
 - Choice 2 → call `associativeMultiplication()`.
 - Choice 3 → terminate.
 - Otherwise → display "Invalid choice".

5. Repeat until the user selects Exit.
6. Stop.

Function: Associative Addition

1. Read positive values m and n .
2. Declare three VLA matrices $A[m][n]$, $B[m][n]$, and $C[m][n]$.
3. Read all three matrices.
4. Calculate:
 - $AB = A + B$
 - $Left = AB + C$
 - $BC = B + C$
 - $Right = A + BC$
5. Display Left and Right.
6. Compare corresponding elements.
7. If every element is equal, state that the associative property is verified.

Function: Associative Multiplication

1. Read positive values m , n , p , and q .
2. Declare:
 - $A[m][n]$
 - $B[n][p]$
 - $C[p][q]$
3. Read all three matrices.
4. Calculate:
 - $AB = A \times B$, having size $m \times p$.
 - $Left = AB \times C$, having size $m \times q$.
 - $BC = B \times C$, having size $n \times q$.
 - $Right = A \times BC$, having size $m \times q$.
5. Display Left and Right.
6. Compare corresponding elements.

7. If all elements are equal, state that the associative property is verified.

❖ C Program

```
#include <stdio.h>

/* Function prototypes */
void associativeAddition(void);
void associativeMultiplication(void);

void readMatrix(int r, int c, int a[r][c]);
void printMatrix(int r, int c, int a[r][c]);

void addMatrix(int r, int c,
               int a[r][c], int b[r][c], int result[r][c]);

void multiplyMatrix(int r1, int c1, int c2,
                   int a[r1][c1],
                   int b[c1][c2],
                   int result[r1][c2]);

int compareMatrix(int r, int c,
                  int a[r][c], int b[r][c]);

int main(void)
{
    int choice;
```

```
do
{
    printf("\n===== MATRIX MENU =====\n");
    printf("1. Verify associative property of matrix addition\n");
    printf("2. Verify associative property of matrix multiplication\n");
    printf("3. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            associativeAddition();
            break;

        case 2:
            associativeMultiplication();
            break;

        case 3:
            printf("Exiting program...\n");
            break;

        default:
            printf("Invalid choice. Please enter 1, 2 or 3.\n");
    }
}
```

```
    } while (choice != 3);

    return 0;
}

/* Function to read a matrix */
void readMatrix(int r, int c, int a[r][c])
{
    int i, j;

    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }
}
```

```
/* Function to display a matrix */
void printMatrix(int r, int c, int a[r][c])
{
    int i, j;

    for (i = 0; i < r; i++)
    {
```

```
        for (j = 0; j < c; j++)
        {
            printf("%d\t", a[i][j]);
        }
        printf("\n");
    }
}
```

```
/* Function to add two matrices */
void addMatrix(int r, int c,
               int a[r][c], int b[r][c], int result[r][c])
{
    int i, j;

    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            result[i][j] = a[i][j] + b[i][j];
        }
    }
}
```

```
/* Function to multiply two matrices */
void multiplyMatrix(int r1, int c1, int c2,
                   int a[r1][c1],
```

```

        int b[c1][c2],
        int result[r1][c2])
{
    int i, j, k;

    for (i = 0; i < r1; i++)
    {
        for (j = 0; j < c2; j++)
        {
            result[i][j] = 0;

            for (k = 0; k < c1; k++)
            {
                result[i][j] += a[i][k] * b[k][j];
            }
        }
    }
}

```

/* Function to compare two matrices */

```

int compareMatrix(int r, int c,
        int a[r][c], int b[r][c])
{
    int i, j;

    for (i = 0; i < r; i++)
    {

```

```

        for (j = 0; j < c; j++)
        {
            if (a[i][j] != b[i][j])
            {
                return 0;
            }
        }
    }

    return 1;
}

```

```

/* Operation 1: Associative property of matrix addition */
void associativeAddition(void)
{
    int m, n;

    printf("\n--- Associative Property of Matrix Addition ---\n");

    do
    {
        printf("Enter number of rows (m): ");
        scanf("%d", &m);

        if (m <= 0)
            printf("Rows must be positive.\n");
    }
}

```

```

} while (m <= 0);

do
{
    printf("Enter number of columns (n): ");
    scanf("%d", &n);

    if (n <= 0)
        printf("Columns must be positive.\n");

} while (n <= 0);

/* Variable-length arrays */
int A[m][n];
int B[m][n];
int C[m][n];

int AB[m][n];
int BC[m][n];

int Left[m][n];
int Right[m][n];

printf("\nEnter elements of matrix A:\n");
readMatrix(m, n, A);

printf("\nEnter elements of matrix B:\n");
readMatrix(m, n, B);

```

```
printf("\nEnter elements of matrix C:\n");
```

```
readMatrix(m, n, C);
```

```
/* (A + B) + C */
```

```
addMatrix(m, n, A, B, AB);
```

```
addMatrix(m, n, AB, C, Left);
```

```
/* A + (B + C) */
```

```
addMatrix(m, n, B, C, BC);
```

```
addMatrix(m, n, A, BC, Right);
```

```
printf("\n(A + B) + C =\n");
```

```
printMatrix(m, n, Left);
```

```
printf("\nA + (B + C) =\n");
```

```
printMatrix(m, n, Right);
```

```
if (compareMatrix(m, n, Left, Right))
```

```
{
```

```
    printf("\nBoth matrices are equal.\n");
```

```
    printf("Hence, the associative property of matrix addition is verified.\n");
```

```
}
```

```
else
```

```
{
```

```
    printf("\nBoth matrices are not equal.\n");
```

```
    printf("Associative property is not verified.\n");
```

```
}
```

```
}
```

```
/* Operation 2: Associative property of matrix multiplication */
```

```
void associativeMultiplication(void)
```

```
{
```

```
    int m, n, p, q;
```

```
    printf("\n--- Associative Property of Matrix Multiplication ---\n");
```

```
    do
```

```
    {
```

```
        printf("Enter m: ");
```

```
        scanf("%d", &m);
```

```
        if (m <= 0)
```

```
            printf("m must be positive.\n");
```

```
    } while (m <= 0);
```

```
    do
```

```
    {
```

```
        printf("Enter n: ");
```

```
        scanf("%d", &n);
```

```
        if (n <= 0)
```

```
            printf("n must be positive.\n");
```

```
} while (n <= 0);
```

```
do
```

```
{
```

```
    printf("Enter p: ");
```

```
    scanf("%d", &p);
```

```
    if (p <= 0)
```

```
        printf("p must be positive.\n");
```

```
} while (p <= 0);
```

```
do
```

```
{
```

```
    printf("Enter q: ");
```

```
    scanf("%d", &q);
```

```
    if (q <= 0)
```

```
        printf("q must be positive.\n");
```

```
} while (q <= 0);
```

```
/* Variable-length arrays */
```

```
int A[m][n];
```

```
int B[n][p];
```

```
int C[p][q];
```

```
/* Intermediate matrices */
```

```

int AB[m][p];

int BC[n][q];


/* Final matrices */

int Left[m][q];
int Right[m][q];


printf("\nEnter elements of matrix A (%d x %d):\n", m, n);
readMatrix(m, n, A);


printf("\nEnter elements of matrix B (%d x %d):\n", n, p);
readMatrix(n, p, B);


printf("\nEnter elements of matrix C (%d x %d):\n", p, q);
readMatrix(p, q, C);


/* (A x B) x C */
multiplyMatrix(m, n, p, A, B, AB);
multiplyMatrix(m, p, q, AB, C, Left);


/* A x (B x C) */
multiplyMatrix(n, p, q, B, C, BC);
multiplyMatrix(m, n, q, A, BC, Right);


printf("\n(A x B) x C =\n");
printMatrix(m, q, Left);


printf("\nA x (B x C) =\n");

```

```

printMatrix(m, q, Right);

if (compareMatrix(m, q, Left, Right))
{
    printf("\nBoth matrices are equal.\n");
    printf("Hence, the associative property of matrix multiplication is verified.\n");
}
else
{
    printf("\nBoth matrices are not equal.\n");
    printf("Associative property is not verified.\n");
}
}

```

❖ Test-Case Table

Test	Choice	Dimensions	Input	Expected Result
1	1	m=2, n=2	Three valid 2×2 matrices	$(A+B)+C = A+(B+C)$
2	1	m=1, n=1	A=5, B=10, C=15	Both sides = 30
3	1	m=0	Invalid dimension	Ask for positive value again
4	2	m=n=p=q=2	Three 2×2 matrices	$(A \times B) \times C = A \times (B \times C)$
5	2	m=2, n=3, p=2, q=3	Compatible rectangular matrices	Both sides are equal 2×3 matrices
6	2	Any dimension ≤ 0	Invalid dimension	Ask for positive value again
7	4	—	Invalid menu choice	Display error message
8	3	—	Exit	Program terminates

❖ Compile, Execute and Evaluate

```
Session Actions Edit View Help
(kali@kali)-[~/Desktop/exp33]
$ vi array6.c

(kali@kali)-[~/Desktop/exp33]
$ gcc array6.c -o array6

(kali@kali)-[~/Desktop/exp33]
$ ./array6

===== MATRIX MENU =====
1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit
Enter your choice: 1

— Associative Property of Matrix Addition —
Enter number of rows (m): 2
Enter number of columns (n): 2

Enter elements of matrix A:
1 2 3 4

Enter elements of matrix B:
4 3 2 1

Enter elements of matrix C:
5 6 7 8

(A + B) + C =
10      11
12      13

A + (B + C) =
10      11
12      13

Both matrices are equal.
Hence, the associative property of matrix addition is verified.
```

===== MATRIX MENU =====

1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit

Enter your choice: 1

— Associative Property of Matrix Addition —

Enter number of rows (m): 1

Enter number of columns (n): 1

Enter elements of matrix A:

5

Enter elements of matrix B:

10

Enter elements of matrix C:

15

$(A + B) + C =$

30

$A + (B + C) =$

30

Both matrices are equal.

Hence, the associative property of matrix addition is verified.

===== MATRIX MENU =====

1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit

Enter your choice: 1

— Associative Property of Matrix Addition —

Enter number of rows (m): 0

Rows must be positive.

Enter number of rows (m): 1

Enter number of columns (n): 1

Enter elements of matrix A:

3

Enter elements of matrix B:

4

Enter elements of matrix C:

5

$(A + B) + C =$

12

$A + (B + C) =$

12

Both matrices are equal.

Hence, the associative property of matrix addition is verified.

===== MATRIX MENU =====

1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit

Enter your choice: 2

— Associative Property of Matrix Multiplication —

Enter m: 2

Enter n: 2

Enter p: 2

Enter q: 2

Enter elements of matrix A (2 x 2):

2 3 4 5

Enter elements of matrix B (2 x 2):

6 7 8 9

Enter elements of matrix C (2 x 2):

1 2 3 4

$(A \times B) \times C =$

159 236
283 420

$A \times (B \times C) =$

159 236
283 420

Both matrices are equal.

Hence, the associative property of matrix multiplication is verified.

===== MATRIX MENU =====

1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit

Enter your choice: 2

— Associative Property of Matrix Multiplication —

Enter m: 2

Enter n: 3

Enter p: 2

Enter q: 3

Enter elements of matrix A (2 x 3):

1 2 3
4 5 6

Enter elements of matrix B (3 x 2):

3 2 1
6 5 4

Enter elements of matrix C (2 x 3):

7 8 9
9 8 7

$(A \times B) \times C =$

374 368 362
887 872 857

$A \times (B \times C) =$

374	368	362
887	872	857

Both matrices are equal.

Hence, the associative property of matrix multiplication is verified.

===== MATRIX MENU =====

1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit

Enter your choice: 2

— Associative Property of Matrix Multiplication —

Enter m: 0

m must be positive.

Enter m: 1

Enter n: 1

Enter p: 1

Enter q: 1

Enter elements of matrix A (1 x 1):

2

Enter elements of matrix B (1 x 1):

3

Enter elements of matrix C (1 x 1):

4

$(A \times B) \times C =$

24

$A \times (B \times C) =$

24

Both matrices are equal.

Hence, the associative property of matrix multiplication is verified.

```
===== MATRIX MENU =====  
1. Verify associative property of matrix addition  
2. Verify associative property of matrix multiplication  
3. Exit  
Enter your choice: 5  
Invalid choice. Please enter 1, 2 or 3.
```

```
===== MATRIX MENU =====  
1. Verify associative property of matrix addition  
2. Verify associative property of matrix multiplication  
3. Exit  
Enter your choice: 3  
Exiting program...
```

```
(kali@kali)-[~/Desktop/exp33]  
$
```